MeterMind — Model 1: Rule-Based DP Reorderer

This notebook implements the **rule-based baseline** for MeterMind.

Given an expanded archaic prose line (e.g. `Thou feedest thy light's own flame with fuel that is self-substantial`), the DP reorderer finds the word ordering that **best matches the iambic pentameter stress pattern** — purely through symbolic constraint-solving, no learning involved.

Why DP and not greedy?

A greedy approach fills each stress position with the first matching word it finds. This can paint itself into a corner — grabbing a word early that would have been more useful later. Dynamic programming instead considers **all possible orderings** and finds the globally optimal one.

Pipeline

```
training_pairs.csv → stress extraction (CMU) → DP reorder → evaluate (MA, SP, G)
```

0. Install dependencies

```
%pip install pronouncing sentence-transformers --quiet
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 939.7/939.7 kB 3.8 MB/s eta 0:00:00
```

1. Imports

```
import re
import io
import math
import pronouncing
import pandas as pd
import matplotlib.pyplot as plt
from google.colab import files
from sentence_transformers import SentenceTransformer, util

print('Imports ready.')
```

```
Imports ready.
```

2. Load dataset

```
# Upload training_pairs.csv when prompted
uploaded = files.upload()
filename = list(uploaded.keys())[0]

df = pd.read_csv(io.StringIO(uploaded[filename].decode('utf-8')))

# Column names from dataset_generation.ipynb are 'input' and 'target'
pairs = list(zip(df['input'].tolist(), df['target'].tolist()))

print(f'Loaded {len(pairs):,} pairs')
print(f'\nExample:')
print(f'  INPUT  : {pairs[0][0]}')
print(f'  TARGET : {pairs[0][1]}')
```

```
Choose Files training_pairs (1).csv
training_pairs (1).csv(text/csv) - 196446 bytes, last modified: 5/10/2026 - 100% done
Saving training_pairs (1).csv to training_pairs (1).csv
Loaded 1,455 pairs

Example:
  INPUT  : His tender heir might bear unto all men his memory most cherished and immortal.
  TARGET : His tender heir might bear his memory:
```

3. Stress extraction

Maps each word to its syllable stress pattern using the CMU Pronouncing Dictionary.

- **Multisyllabic words** (e.g. `compare` → `[0,1]`) have a fixed stress pattern.
- **Monosyllabic words marked as stressed** (e.g. `shall`, `thee`, `day`) are treated as **flexible** — they can sit in either a stressed or unstressed position. This reflects how poets naturally use function words.

```
# ── Shared metric utilities ──────────────────────────────────────────────
# Upload meter_utils.py when prompted, then eval_metrics.py, then imports tokenize, get_stress,
# n_syllables, is_flexible, metrical_accuracy, semantic_preservation,
# grammaticality, load_sp_model, load_gpt2, and IAMBIC_TEMPLATE.
from google.colab import files as _fu
print('Please upload meter_utils.py')
_fu.upload()
from meter_utils import *
print('Please upload eval_metrics.py')
_fu.upload()
from eval_metrics import *

# Sanity check
test_words = ['shall', 'i', 'compare', 'thee', 'to', 'a', 'summers', 'day', 'feedest', 'flame']
print(f"{'word':12} syllables  stress      flexible")
print("-" * 45)
for w in test_words:
    print(f"{w:12} {n_syllables(w):<10} {str(get_stress(w)):12} {is_flexible(w)}")

```

```
Please upload meter_utils.py
Choose Files meter_utils.py
meter_utils.py(n/a) - 5861 bytes, last modified: 5/10/2026 - 100% done
Saving meter_utils.py to meter_utils.py
Please upload eval_metrics.py
Choose Files eval_metrics.py
eval_metrics.py(n/a) - 3273 bytes, last modified: 5/10/2026 - 100% done
Saving eval_metrics.py to eval_metrics.py
word         syllables  stress      flexible
---------------------------------------------
shall        1          [0, 1]       True
i            1          [0, 1]       True
compare      2          [0, 1]       False
thee         1          [0, 1]       True
to           1          [0, 1]       True
a            1          [0]          False
summers      2          [1, 0]       False
day          1          [0, 1]       True
feedest      2          [1, 0]       False
flame        1          [0, 1]       True
```

4. DP Reorderer

How it works

Iambic pentameter has 10 syllable positions: `[0,1,0,1,0,1,0,1,0,1]` (unstressed, stressed, alternating).

The DP solves this as an **assignment problem**:

- **State:** `(syllable_position, set_of_words_used_so_far)`
- **Transition:** try placing each unused word next; score how well its stress pattern matches the template at the current position
- **Goal:** find the word ordering that maximises total stress matches

Because the number of words per line is small (~10-20), we use **bitmask DP** — each word is represented as a bit in an integer, making state representation compact and fast.

Scoring

Each syllable earns 1 point if it matches the iambic template. Flexible monosyllables earn 1 point in any position. The DP maximises total points across all 10 template positions.

```
IAMBIC_TEMPLATE = [0, 1, 0, 1, 0, 1, 0, 1, 0, 1]   # 10 positions: unstressed-STRESSED x5
MAX_TEMPLATE_LEN = len(IAMBIC_TEMPLATE)

```

```python

def word_syllable_score(word, start_pos):
    """
    Score how well `word` fits into the iambic template starting at `start_pos`.
    Returns (score, syllables_consumed).
    """
    stress = get_stress(word)
    score  = 0

    if is_flexible(word):
        # Flexible monosyllable: scores 1 only within template, but always consumes 1 syllable
        score = 1 if start_pos < MAX_TEMPLATE_LEN else 0
        syllables_consumed = 1
    else:
        syllables_consumed = 0
        for i, s in enumerate(stress):
            pos = start_pos + i
            if pos < MAX_TEMPLATE_LEN:
                if s == IAMBIC_TEMPLATE[pos]:
                    score += 1
            syllables_consumed += 1   # always consume, even past template end

    return score, max(syllables_consumed, 1)


def dp_reorder(words):
    """
    Find the word ordering that maximises iambic stress match using bitmask DP.

    State: (syllable_position, bitmask_of_used_words)
    Value: best score achievable from this state

    Returns the reordered word list.
    """
    # Cap at 20 words to keep DP tractable (2^20 = 1M states)
    # Words beyond 20 are appended at the end unchanged
    cap      = min(len(words), 20)
    core     = words[:cap]
    leftover = words[cap:]
    n        = len(core)

    # Pre-compute scores for each (word, start_position) pair
    # score_cache[i][pos] = (score, syllables_consumed) for word i at template position pos
    score_cache = {}
    for i, w in enumerate(core):
        for pos in range(MAX_TEMPLATE_LEN + 1):
            score_cache[(i, pos)] = word_syllable_score(w, pos)

    # DP table: dp[mask][pos] = best score with `mask` words used, at syllable position `pos`
    FULL_MASK = (1 << n) - 1
    NEG_INF   = float('-inf')

    # Use dicts for sparse storage
    dp     = {}   # (mask, pos) -> best_score
    parent = {}   # (mask, pos) -> (prev_mask, prev_pos, word_idx)

    dp[(0, 0)] = 0

    # Process states in order of number of bits set (words placed)
    for num_placed in range(n):
        for (mask, pos), score in list(dp.items()):
            if bin(mask).count('1') != num_placed:
                continue
            # Try placing each unused word
            for i in range(n):
                if mask & (1 << i):
                    continue   # already used

                word_score, syl_consumed = score_cache[(i, pos)]
                new_mask  = mask | (1 << i)
                new_pos   = min(pos + syl_consumed, MAX_TEMPLATE_LEN)
                new_score = score + word_score

                if dp.get((new_mask, new_pos), NEG_INF) < new_score:
                    dp[(new_mask, new_pos)]     = new_score
                    parent[(new_mask, new_pos)] = (mask, pos, i)

```

```
    # Find best final state (all words placed)
    best_score = NEG_INF
    best_state = None
    for pos in range(MAX_TEMPLATE_LEN + 1):
        s = dp.get((FULL_MASK, pos), NEG_INF)
        if s > best_score:
            best_score = s
            best_state = (FULL_MASK, pos)

    # Backtrack to recover word order
    if best_state is None or best_state not in parent:
        return words   # fallback: return original order

    order = []
    state = best_state
    while state in parent:
        prev_mask, prev_pos, word_idx = parent[state]
        order.append(word_idx)
        state = (prev_mask, prev_pos)
    order.reverse()

    reordered = [core[i] for i in order] + leftover
    return reordered


# Quick test
test_line = "thou feedest thy light own flame with fuel that is self substantial"
words     = tokenize(test_line)
result    = dp_reorder(words)
print('Input    :', ' '.join(words))
print('Reordered:', ' '.join(result))
```

```
Input    : thou feedest thy light own flame with fuel that is self substantial
Reordered: thou feedest thy light own flame fuel with that is self substantial
```

4a. Sample DP reorderings

Upload your `training_pairs.csv` here to sanity-check the reorderer on a handful of real pairs before running the full pipeline.

```
# ── Sample DP reorderings on a few real pairs ──────────────────────────────
import io
from google.colab import files as _files

_uploaded = _files.upload()
_filename = list(_uploaded.keys())[0]
_preview_df = pd.read_csv(io.StringIO(_uploaded[_filename].decode('utf-8')))

N_PREVIEW = 5   # change to see more rows

print(f"Showing {N_PREVIEW} sample reorderings\n")
print(f"Template : {IAMBIC_TEMPLATE}")
print(f"{'─'*70}")

for i, (prose_input, target) in enumerate(_preview_df[['input', 'target']].itertuples(index=False)):
    if i >= N_PREVIEW:
        break

    words     = tokenize(prose_input)
    reordered = dp_reorder(words)
    output    = ' '.join(reordered)

    # Quick inline MA (metrics cell may not be run yet)
    correct, syl_pos = 0, 0
    for word in reordered:
        if syl_pos >= len(IAMBIC_TEMPLATE): break
        if is_flexible(word):
            correct += 1; syl_pos += 1
        else:
            for s in get_stress(word):
                if syl_pos >= len(IAMBIC_TEMPLATE): break
                if s == IAMBIC_TEMPLATE[syl_pos]: correct += 1
                syl_pos += 1
    ma = correct / len(IAMBIC_TEMPLATE)

    changed = '(reordered)' if words != reordered else '(unchanged)'
```

```
    print(f"[{i+1}] {changed}")
    print(f"  INPUT    : {prose_input}")
    print(f"  TARGET   : {target}")
    print(f"  OUTPUT   : {output}")
    print(f"  MA       : {ma:.3f}")
    print()

```

Choose Files training_pairs (1).csv

training_pairs (1).csv(text/csv) - 196446 bytes, last modified: 5/10/2026 - 100% done

```
Saving training_pairs (1).csv to training_pairs (1) (1).csv
Showing 5 sample reorderings

Template : [0, 1, 0, 1, 0, 1, 0, 1, 0, 1]
────────────────────────────────────────────────────────────────────────────────
[1] (reordered)
  INPUT    : His tender heir might bear unto all men his memory most cherished and immortal.
  TARGET   : His tender heir might bear his memory:
  OUTPUT   : his tender heir might unto bear all men his memory most cherished and immortal
  MA       : 1.000

[2] (unchanged)
  INPUT    : That tender heir of his might bear his memory forever in their heart and deed.
  TARGET   : His tender heir might bear his memory:
  OUTPUT   : that tender heir of his might bear his memory forever in their heart and deed
  MA       : 1.000

[3] (reordered)
  INPUT    : His memory, which his tender heir might bear with reverence, shall endure through ages.
  TARGET   : His tender heir might bear his memory:
  OUTPUT   : his which his tender heir might bear with memory reverence shall endure through ages
  MA       : 1.000

[4] (reordered)
  INPUT    : Thou art thyself thy foe, and to thy sweet self thou art too cruel in thy dealings.
  TARGET   : Thyself thy foe, to thy sweet self too cruel:
  OUTPUT   : thou art thy foe and to thy sweet self thyself thou art too cruel in thy dealings
  MA       : 1.000

[5] (unchanged)
  INPUT    : To thy own sweet self thou dost prove too cruel, being thyself thy very foe.
  TARGET   : Thyself thy foe, to thy sweet self too cruel:
  OUTPUT   : to thy own sweet self thou dost prove too cruel being thyself thy very foe
  MA       : 1.000
```

5. Evaluation metrics

Three metrics, matching the MeterMind evaluation framework:

Metric	What it measures	How
MA — Metrical Accuracy	How well the output matches iambic stress pattern	CMU stress vs template
SP — Semantic Preservation	How much meaning is retained	Cosine similarity of SentenceTransformer embeddings
G — Grammaticality	How fluent/natural the output reads	GPT-2 perplexity (lower = better)

```
# ── Load models (SentenceTransformer + GPT-2) ──────────────────────────────
# meter_utils lazy-loads these on first call, but we trigger them here so the
# download happens before the eval loop rather than mid-run.
load_sp_model()
load_gpt2()
print('All metrics ready.')

```

```
Loading SentenceTransformer (first run downloads ~90 MB)...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
modules.json: 100%  349/349 [00:00<00:00, 22.0kB/s]
config_sentence_transformers.json: 100%  116/116 [00:00<00:00, 10.8kB/s]
README.md: 10.5k/? [00:00<00:00, 901kB/s]
sentence_bert_config.json: 100%  53.0/53.0 [00:00<00:00, 4.55kB/s]
config.json: 100%  612/612 [00:00<00:00, 51.5kB/s]
model.safetensors: 100%  90.9M/90.9M [00:02<00:00, 451MB/s]
Loading weights: 100%  103/103 [00:00<00:00, 463.23it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
tokenizer_config.json: 100%  350/350 [00:00<00:00, 32.6kB/s]
vocab.txt: 232k/? [00:00<00:00, 5.75MB/s]
tokenizer.json: 466k/? [00:00<00:00, 14.2MB/s]
special_tokens_map.json: 100%  112/112 [00:00<00:00, 9.76kB/s]
config.json: 100%  190/190 [00:00<00:00, 15.8kB/s]
Loading GPT-2 (first run downloads ~500 MB)...
tokenizer_config.json: 100%  26.0/26.0 [00:00<00:00, 2.16kB/s]
vocab.json: 1.04M/? [00:00<00:00, 1.72MB/s]
merges.txt: 456k/? [00:00<00:00, 979kB/s]
tokenizer.json: 1.36M/? [00:00<00:00, 3.88MB/s]
config.json: 100%  665/665 [00:00<00:00, 69.1kB/s]
model.safetensors: 100%  548M/548M [00:06<00:00, 119MB/s]
Loading weights: 100%  148/148 [00:00<00:00, 525.96it/s, Materializing param=transformer.wte.weight]
GPT2LMHeadModel LOAD REPORT from: gpt2
Key                  | Status     |  |
---------------------+------------+--+-
h.{0...11}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
generation_config.json: 100%  124/124 [00:00<00:00, 12.0kB/s]
All metrics ready.
```

5a. Sanity-check: Metrical Accuracy on uploaded sample pairs

Upload your `training_pairs.csv` here to spot-check `metrical_accuracy` on a handful of rows — before running the full pipeline. We print the per-syllable alignment so you can see exactly where matches and misses fall.

```
# ── Upload a sample of training_pairs.csv and check MA ──────────────────────
import io
from google.colab import files as colab_files

sample_uploaded = colab_files.upload()
sample_filename = list(sample_uploaded.keys())[0]
sample_df = pd.read_csv(io.StringIO(sample_uploaded[sample_filename].decode('utf-8')))

N_SAMPLE = 10   # how many pairs to inspect; change as needed
sample_pairs = list(zip(sample_df['input'], sample_df['target']))[:N_SAMPLE]

```

```python
def ma_debug(words, template=IAMBIC_TEMPLATE):
    """
    Like metrical_accuracy() but prints a per-syllable alignment table
    so you can verify the logic visually.
    """
    correct = 0
    syl_pos = 0
    rows = []

    for word in words:
        if syl_pos >= len(template):
            rows.append((word, '-', '-', '-', 'OVERFLOW'))
            continue
        if is_flexible(word):
            rows.append((word, 'flex', template[syl_pos], '✓', syl_pos))
            correct += 1
            syl_pos += 1
        else:
            stress = get_stress(word)
            for s in stress:
                if syl_pos >= len(template):
                    break
                match = s == template[syl_pos]
                rows.append((word, s, template[syl_pos], '✓' if match else '✗', syl_pos))
                if match:
                    correct += 1
                syl_pos += 1

    ma = correct / len(template)
    return ma, rows

print(f"Checking {N_SAMPLE} pairs — template: {IAMBIC_TEMPLATE}\n")
print(f"{'#':<3} {'MA':>5}  Input line")
print('-' * 60)

for idx, (prose_input, target) in enumerate(sample_pairs):
    words = tokenize(prose_input)
    ma, rows = ma_debug(words)
    print(f"\n[{idx+1}] MA={ma:.3f}  input : {prose_input}")
    print(f"{'':7}        target: {target}")
    print(f"  {'word':<14} {'stress':>6}  {'tmpl':>4}  {'match':>5}  pos")
    print(f"  {'-'*14} {'-'*6}  {'-'*4}  {'-'*5}  ---")
    for word, stress, tmpl, match, pos in rows:
        print(f"  {word:<14} {str(stress):>6}  {str(tmpl):>4}  {match:>5}  {pos}")

```

Choose Files training_pairs (1).csv

training_pairs (1).csv(text/csv) - 196446 bytes, last modified: 5/10/2026 - 100% done

```
Saving training_pairs (1).csv to training_pairs (1) (2).csv
Checking 10 pairs — template: [0, 1, 0, 1, 0, 1, 0, 1, 0, 1]

#      MA  Input line
------------------------------------------------------------

[1] MA=0.800  input : His tender heir might bear unto all men his memory most cherished and immortal.
               target: His tender heir might bear his memory:
  word           stress  tmpl  match  pos
  -------------- ------  ----  -----  ---
  his              flex     0      ✓  0
  tender              1     1      ✓  1
  tender              0     0      ✓  2
  heir             flex     1      ✓  3
  might            flex     0      ✓  4
  bear             flex     1      ✓  5
  unto                1     0      ✗  6
  unto                0     1      ✗  7
  all              flex     0      ✓  8
  men              flex     1      ✓  9
  his                 -     -      -  OVERFLOW
  memory              -     -      -  OVERFLOW
  most                -     -      -  OVERFLOW
  cherished           -     -      -  OVERFLOW
  and                 -     -      -  OVERFLOW
  immortal            -     -      -  OVERFLOW

[2] MA=1.000  input : That tender heir of his might bear his memory forever in their heart and deed.
               target: His tender heir might bear his memory:
  word           stress  tmpl  match  pos
  -------------- ------  ----  -----  ---
  that             flex     0      ✓  0
  tender              1     1      ✓  1
  tender              0     0      ✓  2
  heir             flex     1      ✓  3
  of               flex     0      ✓  4
  his              flex     1      ✓  5
  might            flex     0      ✓  6
  bear             flex     1      ✓  7
  his              flex     0      ✓  8
  memory              1     1      ✓  9
  forever             -     -      -  OVERFLOW
  in                  -     -      -  OVERFLOW
  their               -     -      -  OVERFLOW
  heart               -     -      -  OVERFLOW
  and                 -     -      -  OVERFLOW
  deed                -     -      -  OVERFLOW

[3] MA=0.700  input : His memory, which his tender heir might bear with reverence, shall endure through ages.
               target: His tender heir might bear his memory:
  word           stress  tmpl  match  pos
  -------------- ------  ----  -----  ---
  his              flex     0      ✓  0
  memory              1     1      ✓  1
  memory              0     0      ✓  2
  memory              0     1      ✗  3
  which            flex     0      ✓  4
  his              flex     1      ✓  5
  tender              1     0      ✗  6
  tender              0     1      ✗  7
  heir             flex     0      ✓  8
  might            flex     1      ✓  9
  bear                -     -      -  OVERFLOW
  with                -     -      -  OVERFLOW
  reverence           -     -      -  OVERFLOW
  shall               -     -      -  OVERFLOW
  endure              -     -      -  OVERFLOW
  through             -     -      -  OVERFLOW
  ages                -     -      -  OVERFLOW

[4] MA=0.900  input : Thou art thyself thy foe, and to thy sweet self thou art too cruel in thy dealings.
               target: Thyself thy foe, to thy sweet self too cruel:
  word           stress  tmpl  match  pos
  -------------- ------  ----  -----  ---
  thou             flex     0      ✓  0
  art              flex     1      ✓  1
  thyself             1     0      ✗  2
  thyself             1     1      ✓  3
  thy              flex     0      ✓  4
  foe              flex     1      ✓  5
  and                 0     0      ✓  6
  to               flex     1      ✓  7
```

```
  thy              flex     0      ✓  8
  sweet            flex     1      ✓  9
  self                -     -      -  OVERFLOW
  thou                -     -      -  OVERFLOW
  art                 -     -      -  OVERFLOW
  too                 -     -      -  OVERFLOW
  cruel               -     -      -  OVERFLOW
  in                  -     -      -  OVERFLOW
  thy                 -     -      -  OVERFLOW
  dealings            -     -      -  OVERFLOW

[5] MA=1.000  input : To thy own sweet self thou dost prove too cruel, being thyself thy very foe.
              target: Thyself thy foe, to thy sweet self too cruel:
  word           stress  tmpl  match  pos
  -------------- ------  ----  -----  ---
  to               flex     0      ✓  0
  thy              flex     1      ✓  1
  own              flex     0      ✓  2
  sweet            flex     1      ✓  3
  self             flex     0      ✓  4
  thou             flex     1      ✓  5
  dost             flex     0      ✓  6
  prove            flex     1      ✓  7
  too              flex     0      ✓  8
  cruel               1     1      ✓  9
  being               -     -      -  OVERFLOW
  thyself             -     -      -  OVERFLOW
  thy                 -     -      -  OVERFLOW
  very                -     -      -  OVERFLOW
  foe                 -     -      -  OVERFLOW

[6] MA=0.800  input : Thyself thy foe—thus to thy sweet self art thou too cruel in this manner.
              target: Thyself thy foe, to thy sweet self too cruel:
  word           stress  tmpl  match  pos
  -------------- ------  ----  -----  ---
  thyself             1     0      X  0
  thyself             1     1      ✓  1
  thy              flex     0      ✓  2
  foethus             0     1      X  3
  to               flex     0      ✓  4
  thy              flex     1      ✓  5
  sweet            flex     0      ✓  6
  self             flex     1      ✓  7
  art              flex     0      ✓  8
  thou             flex     1      ✓  9
  too                 -     -      -  OVERFLOW
  cruel               -     -      -  OVERFLOW
  in                  -     -      -  OVERFLOW
  this                -     -      -  OVERFLOW
  manner              -     -      -  OVERFLOW

[7] MA=0.500  input : Where doth all the treasure of thy lusty days remain concealed?
              target: Where all the treasure of thy lusty days;
  word           stress  tmpl  match  pos
  -------------- ------  ----  -----  ---
  where            flex     0      ✓  0
  doth             flex     1      ✓  1
  all              flex     0      ✓  2
  the                 0     1      X  3
  treasure            1     0      X  4
  treasure            0     1      X  5
  of               flex     0      ✓  6
  thy              flex     1      ✓  7
  lusty               1     0      X  8
  lusty               0     1      X  9
  days                -     -      -  OVERFLOW
  remain              -     -      -  OVERFLOW
  concealed           -     -      -  OVERFLOW

[8] MA=0.500  input : All the treasure of thy lusty days — where, prithee, doth it lie?
              target: Where all the treasure of thy lusty days;
  word           stress  tmpl  match  pos
  -------------- ------  ----  -----  ---
  all              flex     0      ✓  0
  the                 0     1      X  1
  treasure            1     0      X  2
  treasure            0     1      X  3
  of               flex     0      ✓  4
  thy              flex     1      ✓  5
  lusty               1     0      X  6
  lusty               0     1      X  7
  days             flex     0      ✓  8
  where            flex     1      ✓  9
  prithee             -     -      -  OVERFLOW
  doth                -     -      -  OVERFLOW
```

6. Run DP reorderer on full dataset

```
[9] MA=0.700  input : Where hath all the treasure of thy most lusty days been kept and stored?
```

```
# ── Step 1 & 2: DP reordering + Metrical Accuracy (pure Python, fast) ──────────
from tqdm.notebook import tqdm

rows = []
for prose_input, target in tqdm(pairs, desc='Reordering + MA'):
    words     = tokenize(prose_input)
    reordered = dp_reorder(words)
    output    = ' '.join(reordered)
    rows.append({
        'input':  prose_input,
        'target': target,
        'output': output,
        'MA':     round(metrical_accuracy(reordered), 4),
    })

print(f'{len(rows):,} lines reordered.')

```

```
Reordering + MA: 100%                                    1455/1455 [1:52:15<00:00,  2.70s/it]
1,455 lines reordered.
```

```
# ── Step 3: Semantic Preservation (batched encode) ──────────────────────────
from sentence_transformers import util

inputs_clean  = [' '.join(tokenize(r['input'])) for r in rows]
outputs_clean = [r['output'] for r in rows]

print(f'Encoding {len(rows):,} input+output pairs...')
embs = load_sp_model().encode(
    inputs_clean + outputs_clean,
    convert_to_tensor=True,
    show_progress_bar=True,
)

n = len(rows)
for i, r in enumerate(rows):
    r['SP'] = round(float(max(0.0, util.cos_sim(embs[i], embs[n + i]))), 4)

print('SP done.')

```

```
Encoding 1,455 input+output pairs...
Batches: 100%                                            91/91 [00:35<00:00,  3.11it/s]
SP done.
```

```
# ── Step 4: Grammaticality (batched GPT-2, per-sequence loss) ──────────────
import torch, math
from tqdm.notebook import tqdm as tqdm_nb

BATCH_SIZE   = 32
gpt2_model, gpt2_tok = load_gpt2()

# GPT-2 has no pad token by default — use eos as pad (standard practice)
if gpt2_tok.pad_token_id is None:
    gpt2_tok.pad_token_id = gpt2_tok.eos_token_id

outputs_list = [r['output'] for r in rows]
g_scores     = []

for batch_start in tqdm_nb(range(0, len(outputs_list), BATCH_SIZE), desc='Grammaticality'):
    batch = outputs_list[batch_start : batch_start + BATCH_SIZE]
    enc   = gpt2_tok(
        batch,
        return_tensors='pt',
        padding=True,
        truncation=True,
        max_length=128,
    )
    input_ids = enc['input_ids']
    # Labels: same as input_ids but pad positions set to -100 (ignored by loss)
```

```
    labels = input_ids.clone()
    labels[input_ids == gpt2_tok.pad_token_id] = -100

    with torch.no_grad():
        logits = gpt2_model(**enc).logits   # (batch, seq_len, vocab)

    # Per-sequence loss: shift so token i predicts token i+1
    shift_logits = logits[:, :-1, :].contiguous()
    shift_labels = labels[:, 1:].contiguous()

    loss_fn     = torch.nn.CrossEntropyLoss(ignore_index=-100, reduction='none')
    token_losses = loss_fn(
        shift_logits.view(-1, shift_logits.size(-1)),
        shift_labels.view(-1),
    ).view(len(batch), -1)   # (batch, seq_len-1)

    mask       = (shift_labels != -100).float()
    seq_losses = (token_losses * mask).sum(-1) / mask.sum(-1).clamp(min=1)

    for loss_val in seq_losses.tolist():
        perplexity = math.exp(loss_val)
        g_scores.append(round(1 / (1 + math.log(perplexity)), 4))

for r, g in zip(rows, g_scores):
    r['G'] = g

print(f'G done. {len(g_scores):,} scores computed.')

```

```
Grammaticality: 100%                                    46/46 [03:26<00:00,  3.49s/it]
G done. 1,455 scores computed.
```

```
# ── Step 5: Assemble results DataFrame ──────────────────────────────────
import pandas as pd

results_df = pd.DataFrame(rows)
print(f'Done. {len(results_df):,} results.')

```

```
Done. 1,455 results.
```

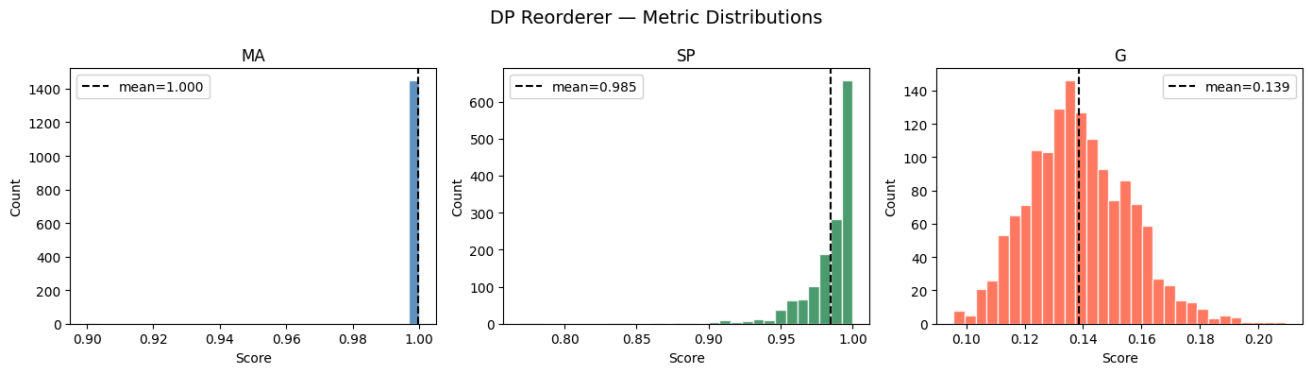
7. Results

```
print('=== DP Reorderer — Aggregate Results ===')
print(f"Metrical Accuracy (MA) : {results_df['MA'].mean():.4f} ± {results_df['MA'].std():.4f}")
print(f"Semantic Preservation (SP): {results_df['SP'].mean():.4f} ± {results_df['SP'].std():.4f}")
print(f"Grammaticality (G)     : {results_df['G'].mean():.4f} ± {results_df['G'].std():.4f}")

print('\n=== Sample outputs ===')
for _, row in results_df.head(5).iterrows():
    print(f"INPUT  : {row['input']}")
    print(f"TARGET : {row['target']}")
    print(f"OUTPUT : {row['output']}")
    print(f"MA={row['MA']:.3f}  SP={row['SP']:.3f}  G={row['G']:.3f}")
    print()
```

```
=== DP Reorderer — Aggregate Results ===
Metrical Accuracy (MA) : 0.9996 ± 0.0064
Semantic Preservation (SP): 0.9847 ± 0.0207
Grammaticality (G)     : 0.1387 ± 0.0175

=== Sample outputs ===
INPUT  : His tender heir might bear unto all men his memory most cherished and immortal.
TARGET : His tender heir might bear his memory:
OUTPUT : his tender heir might unto bear all men his memory most cherished and immortal
MA=1.000  SP=0.994  G=0.115

INPUT  : That tender heir of his might bear his memory forever in their heart and deed.
TARGET : His tender heir might bear his memory:
OUTPUT : that tender heir of his might bear his memory forever in their heart and deed
MA=1.000  SP=1.000  G=0.137

INPUT  : His memory, which his tender heir might bear with reverence, shall endure through ages.
TARGET : His tender heir might bear his memory:
OUTPUT : his which his tender heir might bear with memory reverence shall endure through ages
```

```
MA=1.000  SP=0.987  G=0.119

INPUT  : Thou art thyself thy foe, and to thy sweet self thou art too cruel in thy dealings.
TARGET : Thyself thy foe, to thy sweet self too cruel:
OUTPUT : thou art thy foe and to thy sweet self thyself thou art too cruel in thy dealings
MA=1.000  SP=0.997  G=0.160

INPUT  : To thy own sweet self thou dost prove too cruel, being thyself thy very foe.
TARGET : Thyself thy foe, to thy sweet self too cruel:
OUTPUT : to thy own sweet self thou dost prove too cruel being thyself thy very foe
MA=1.000  SP=1.000  G=0.144
```

```
# Distribution plots
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
fig.suptitle('DP Reorderer — Metric Distributions', fontsize=14)

for ax, metric, colour in zip(axes, ['MA', 'SP', 'G'], ['steelblue', 'seagreen', 'tomato']):
    ax.hist(results_df[metric], bins=30, color=colour, edgecolor='white', alpha=0.85)
    ax.axvline(results_df[metric].mean(), color='black', linestyle='--', linewidth=1.5, label=f"mean={results_df[metric].me
    ax.set_title(metric)
    ax.set_xlabel('Score')
    ax.set_ylabel('Count')
    ax.legend()

plt.tight_layout()
plt.savefig('dp_reorderer_metrics.png', dpi=150)
plt.show()
```



8. Save results

```
from google.colab import files
results_df.to_csv('dp_reorderer_results.csv', index=False)
print('Saved dp_reorderer_results.csv')

# Summary dict — import this into the evaluation notebook to compare all 3 models
import json
summary = {
    'model': 'DP Reorderer',
    'n':     len(results_df),
    'MA':    {'mean': round(float(results_df['MA'].mean()), 4), 'std': round(float(results_df['MA'].std()), 4)},
    'SP':    {'mean': round(float(results_df['SP'].mean()), 4), 'std': round(float(results_df['SP'].std()), 4)},
    'G':     {'mean': round(float(results_df['G'].mean()),  4), 'std': round(float(results_df['G'].std()),  4)},
}
with open('dp_reorderer_summary.json', 'w') as f:
    json.dump(summary, f, indent=2)
print('Saved dp_reorderer_summary.json')

# Download both
files.download('dp_reorderer_results.csv')
files.download('dp_reorderer_summary.json')
files.download('dp_reorderer_metrics.png')
```

```
Saved dp_reorderer_results.csv
Saved dp_reorderer_summary.json
```

Mini Eval — Cross-Model Sample Comparison

Runs the full eval pipeline on a small fixed sample from the test set. Use the same `N_EVAL` and `SEED` across all three notebooks to ensure the comparison is fair.

```
# ── Mini eval ──────────────────────────────────────────────────────────
import random
from tqdm.notebook import tqdm

N_EVAL = 50
SEED   = 42

# Define test split inline (model 1 has no training so this wasn't needed before)
random.seed(SEED)
_unique = sorted({t for _, t in pairs})
random.shuffle(_unique)
_n = len(_unique)
_test_targets = set(_unique[int(_n * 0.9):])
_test_pairs   = [(s, t) for s, t in pairs if t in _test_targets]

# Sample
random.seed(SEED)
_eval_pairs = random.sample(_test_pairs, min(N_EVAL, len(_test_pairs)))

# ── Step 1: Inference + MA ─────────────────────────────────────────────
_rows = []
for _src, _tgt in tqdm(_eval_pairs, desc='DP reorder + MA'):
    _words     = tokenize(_src)
    _reordered = dp_reorder(_words)
    _output    = ' '.join(_reordered)
    _rows.append({
        'input':  _src,
        'target': _tgt,
        'output': _output,
        'MA':     round(metrical_accuracy(_reordered), 4),
    })

# ── Step 2: Semantic Preservation (batched) ────────────────────────────
from sentence_transformers import util as _util

_inputs_clean  = [' '.join(tokenize(r['input'])) for r in _rows]
_outputs_clean = [r['output'] for r in _rows]
_embs = load_sp_model().encode(
    _inputs_clean + _outputs_clean,
    convert_to_tensor=True,
    show_progress_bar=True,
)
_n = len(_rows)
for _i, _r in enumerate(_rows):
    _r['SP'] = round(float(max(0.0, _util.cos_sim(_embs[_i], _embs[_n + _i]))), 4)

# ── Step 3: Grammaticality (batched GPT-2) ─────────────────────────────
import torch as _torch, math as _math

_G_BATCH  = 32
_gm, _gt  = load_gpt2()
if _gt.pad_token_id is None:
    _gt.pad_token_id = _gt.eos_token_id

_out_list = [r['output'] for r in _rows]
_g_scores = []
for _bs in range(0, len(_out_list), _G_BATCH):
    _batch = _out_list[_bs : _bs + _G_BATCH]
    _enc   = _gt(_batch, return_tensors='pt', padding=True, truncation=True, max_length=128)
    _ids   = _enc['input_ids']
    _lbl   = _ids.clone()
    _lbl[_ids == _gt.pad_token_id] = -100
    with _torch.no_grad():
        _lg = _gm(**_enc).logits
    _sl = _lg[:, :-1, :].contiguous()
    _tl = _lbl[:, 1:].contiguous()
```

```
    _fn = _torch.nn.CrossEntropyLoss(ignore_index=-100, reduction='none')
    _tl2 = _fn(_sl.view(-1, _sl.size(-1)), _tl.view(-1)).view(len(_batch), -1)
    _mk = (_tl != -100).float()
    _sq = (_tl2 * _mk).sum(-1) / _mk.sum(-1).clamp(min=1)
    for _lv in _sq.tolist():
        _g_scores.append(round(1 / (1 + _lv), 4))
```